Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет программной инженерии и компьютерной техники
Образовательная программа «Веб-технологии»

О Т Ч Ё Т

по дисциплине «Базы данных и знаний»

Тема: Проектирование и анализ схемы хранения композитных объектов, представляемых на основе pure operation-based CRDT

Обучающийся: Иванов Никита Денисович

Группа: P4110

Срок выполнения: апрель 2026

Санкт-Петербург
2026

Содержание

ВВЕДЕНИЕ **4**

1 ПОСТАНОВКА ЗАДАЧИ И МОДЕЛЬ ДАННЫХ **5**

1.1 Состав репозитория 5

1.2 Фактическая структура операции 5

2 РЕАЛИЗАЦИЯ ВАРИАНТА НА POSTGRESQL **7**

2.1 Назначение PostgreSQL-варианта 7

2.2 Логическая и физическая схема 7

2.3 Целостность и нормализация 8

2.4 Основные сценарии запросов 9

2.5 Фрагмент SQL-схемы 10

2.6 Преимущества и ограничения 11

3 РЕАЛИЗАЦИЯ ВАРИАНТА НА MONGODB **12**

3.1 Назначение MongoDB-варианта 12

3.2 Выбранная документная модель 12

3.3 Индексация и сценарии чтения 13

3.4 Преимущества и ограничения 14

3.5 Фрагмент инициализации MongoDB 14

3.6 Наблюдения по экспериментальным данным 16

4 ИЗМЕРЕНИЯ И РЕЗУЛЬТАТЫ **17**

4.1 Набор команд для анализа 17

4.2 Результаты PostgreSQL 17

4.3 Результаты MongoDB 18

5 ВАРИАНТ НА YTSAURUS **21**

5.1 Общая характеристика платформы 21

5.2 Экспериментальное окружение 22
5.3 Реализованная модель хранения 23
5.4 Генератор данных и сценарии запросов 23
5.5 Результаты замеров времени 25
5.6 Параметры таблицы и объём хранения 25
5.7 Промежуточный вывод 28

ЗАКЛЮЧЕНИЕ **29**

ПРИЛОЖЕНИЕ А. СВОДНЫЕ РЕЗУЛЬТАТЫ POSTGRESQL **32**

ПРИЛОЖЕНИЕ Б. СВОДНЫЕ РЕЗУЛЬТАТЫ MONGODB **34**

ПРИЛОЖЕНИЕ В. СВОДНЫЕ РЕЗУЛЬТАТЫ YTSAURUS **36**

ПРИЛОЖЕНИЕ Г. СРАВНЕНИЕ ПОДХОДОВ ХРАНЕНИЯ **38**

ВВЕДЕНИЕ

В рамках индивидуального задания рассматривается задача проектирования хранилища для композитных объектов, состояние которых определяется журналом операций. Исходная предметная область связана с прототипом коллаборативного календарного приложения, поддерживающего оффлайн-режим, локальные изменения и последующую синхронизацию между репликами.

Актуальность темы обусловлена тем, что при построении систем на основе CRDT необходимо не только определить прикладную модель конфликтов и синхронизации, но и подобрать такую структуру хранения, которая будет одновременно удобной для записи операций, восстановления истории объекта и анализа производительности типовых запросов.

Целью работы является сравнение двух вариантов хранения одной и той же модели операций: в реляционной СУБД PostgreSQL и в документной СУБД MongoDB.

Для достижения поставленной цели решаются следующие задачи:

- определить логическую структуру данных, описывающую реплики, объекты и операции;
- спроектировать схему хранения в PostgreSQL;
- спроектировать схему хранения в MongoDB;
- проанализировать сильные и слабые стороны каждого подхода;
- подготовить основу для дальнейшего анализа индексов и планов выполнения запросов.

ГЛАВА 1. ПОСТАНОВКА ЗАДАЧИ И МОДЕЛЬ ДАННЫХ

Репозиторий выделен как отдельная инфраструктурная часть индивидуального задания и предназначен для исследования нескольких вариантов хранения одного и того же контракта операций CRDT. Практический интерес работы состоит не только в выборе СУБД, но и в сравнении разных способов организации одного и того же operation log.

1.1 Состав репозитория

В текущем состоянии репозиторий включает следующие основные части:

- каталог `postgresql` с `Docker Compose`, миграцией схемы и материалами по анализу запросов;
- каталог `mongodb` с `Docker Compose`, инициализацией коллекции и материалами по `executionStats`;
- каталог `report` с исходным текстом данного отчёта.

Таким образом, работа организована как воспроизводимое исследовательское окружение, в котором каждая СУБД поднимается изолированно и имеет собственные скрипты инициализации.

1.2 Фактическая структура операции

Согласно описанию в `README.md` и специализированных подкаталогах, центральной единицей прикладной модели является операция со следующими полями:

- `opId`;
- `txId`;

- `objectId`;
- `replicaId`;
- `clock`;
- `action`.

Поле `clock` хранит причинно-следственную информацию. Поле `action` содержит полезную нагрузку операции. В обоих вариантах используется одна и та же прикладная структура; различается только физическая организация данных.

ГЛАВА 2. РЕАЛИЗАЦИЯ ВАРИАНТА НА POSTGRESQL

В каталоге `postgresql` реализовано автономное окружение для реляционного моделирования журнала CRDT-операций. Запуск выполняется через `dockercompose-fpostgresql/compose.yamlup-d`, а физическая схема создаётся миграцией `postgresql/initdb/001_schema.sql`.

2.1 Назначение PostgreSQL-варианта

PostgreSQL используется как базовая реляционная СУБД для хранения append-only operation log. Такой вариант удобен для анализа, поскольку позволяет явно задать структуру данных, внешние ключи, ограничения целостности и индексную стратегию, а затем исследовать поведение запросов через `EXPLAIN ANALYZE`.

2.2 Логическая и физическая схема

Текущая реляционная схема включает три таблицы:

- `replicas` — справочник логических реплик;
- `objects` — справочник корневых CRDT-объектов;
- `operations` — неизменяемый журнал операций.

Связи между сущностями заданы явно: одна реплика может породить много операций, один объект может иметь много операций, каждая операция принадлежит ровно одной реплике и ровно одному объекту. При этом отдельная таблица зависимостей между операциями не вводится, поскольку причинность задаётся содержимым поля `clock`.

Физическая схема повторяет логическую модель почти напрямую. Идентификаторы хранятся как `TEXT`, что соответствует строковому формату

библиотеки. Поля `clock` и `action` сохраняются в `JSONB`, поскольку они имеют полуструктурированную природу и на текущем этапе исследования не декомпозируются в набор отдельных реляционных таблиц.

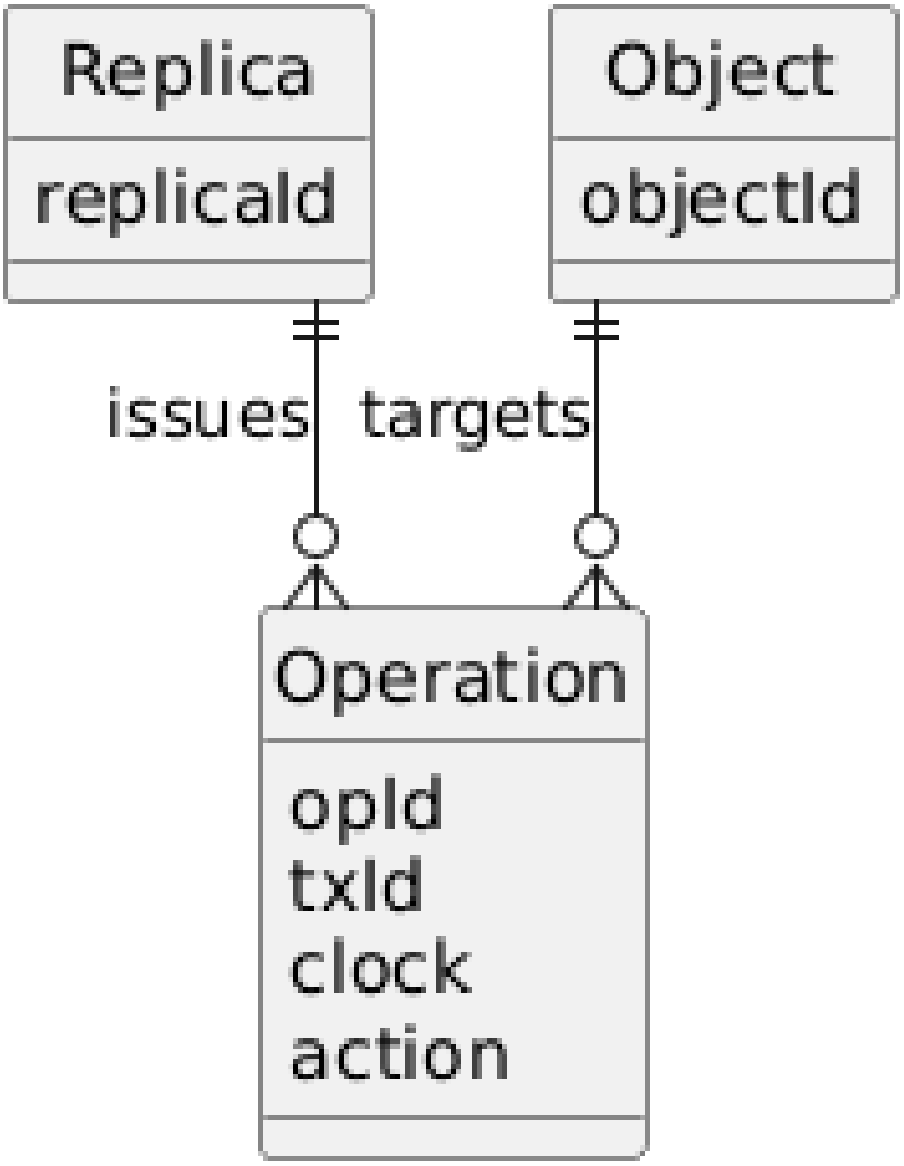

Рисунок 1 – Логическая ER-диаграмма варианта хранения в PostgreSQL

2.3 Целостность и нормализация

Схему PostgreSQL корректно считать частично нормализованной. Каркас из таблиц `replicas`, `objects` и `operations` соответствует обычной реляционной модели и обеспечивает:

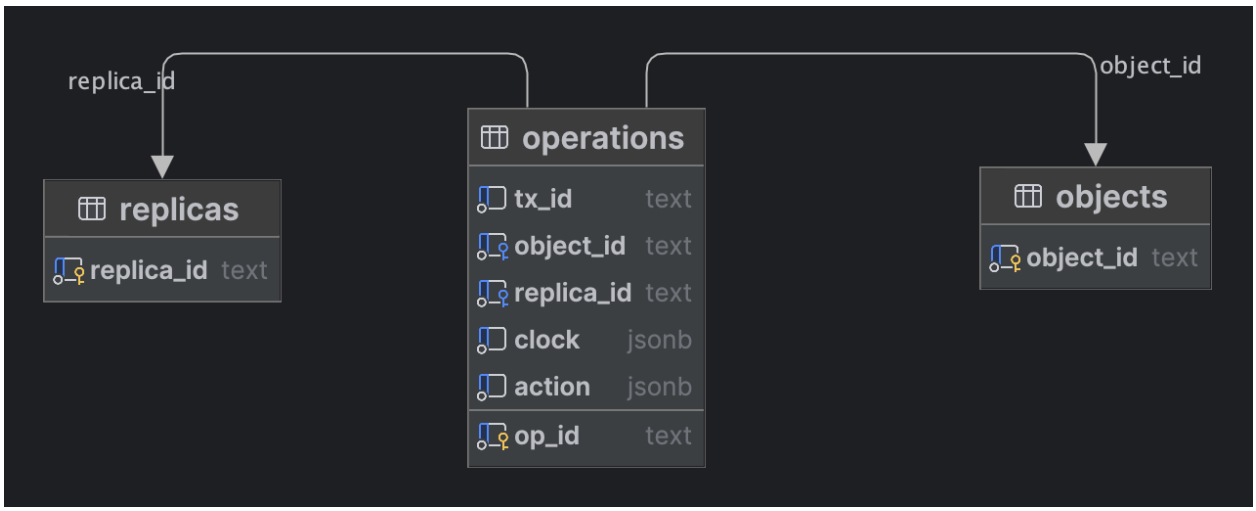

Рисунок 2 – Физическая схема таблиц PostgreSQL

- первичные ключи для всех основных сущностей;
- внешние ключи из `operations` в `replicas` и `objects`;
- `CHECK`-ограничения, контролирующие, что `clock` и `action` являются JSON-объектами;
- проверку наличия поля `type` внутри `action`.

При этом доменная корректность содержимого `clock` и `action` остаётся обязанностью прикладного уровня. Это означает, что PostgreSQL отвечает за структурную и ссылочную целостность, а библиотека — за корректную семантику CRDT-операций.

2.4 Основные сценарии запросов

Из `postgresql/README.md` видно, что в реляционном варианте в фокусе находятся выборки по операциям как самостоятельным записям. Базовые сценарии:

- получение полной истории операций для заданного `objectId`;
- получение операций конкретной транзакции по `txId`;
- получение операций, выпущенных конкретной репликой;

- чтение сохранённых векторных часов для конкретной операции;
- фильтрация по типу действия внутри `action`.

Этот набор запросов показывает, что PostgreSQL ориентирован не только на чтение истории одного объекта, но и на глобальный анализ журнала операций.

2.5 Фрагмент SQL-схемы

Ниже приведён фактический фрагмент миграции, создающей основные таблицы и индексы реляционного варианта:

```sql
CREATE TABLE replicas (
    replica_id TEXT PRIMARY KEY
);

CREATE TABLE objects (
    object_id TEXT PRIMARY KEY
);

CREATE TABLE operations (
    op_id TEXT PRIMARY KEY,
    tx_id TEXT NOT NULL,
    object_id TEXT NOT NULL REFERENCES objects(object_id) ON DELETE CASCADE,
    replica_id TEXT NOT NULL REFERENCES replicas(replica_id),
    clock JSONB NOT NULL,
    action JSONB NOT NULL,
    CONSTRAINT chk_clock_is_object CHECK (jsonb_typeof(clock) = 'object'),
    CONSTRAINT chk_action_is_object CHECK (jsonb_typeof(action) = 'object'),
    CONSTRAINT chk_action_has_type CHECK (action ? 'type')
);

CREATE INDEX idx_operations_object_id
    ON operations (object_id);

CREATE INDEX idx_operations_replica_id
    ON operations (replica_id);

```

```sql
CREATE INDEX idx_operations_tx_id
    ON operations (tx_id);

CREATE INDEX idx_operations_action_type
    ON operations ((action ->> 'type'));

CREATE INDEX idx_operations_clock_gin
    ON operations USING GIN (clock);

CREATE INDEX idx_operations_action_gin
    ON operations USING GIN (action);
```

Листинг 1 – Миграция схемы PostgreSQL

2.6 Преимущества и ограничения

К основным преимуществам PostgreSQL-варианта относятся строгая схема хранения, прозрачная ссылочная целостность, удобство аналитических выборок по `opId`, `txId` и `objectId`, а также естественная поддержка append-only журнала как множества независимых строк.

Ограничения реляционного варианта также явно видны. Во-первых, часть структуры операции остаётся внутри `JSONB`, поэтому схема является лишь частично нормализованной. Во-вторых, длинная история одного объекта требует сортировки и может становиться заметной по стоимости. В-третьих, низкоселективные широкие выборки могут приводить к менее эффективным планам, включая `Seq Scan`.

ГЛАВА 3. РЕАЛИЗАЦИЯ ВАРИАНТА НА MONGODB

В каталоге `mongodb` размещено автономное окружение документной СУБД MongoDB. Запуск выполняется через `dockercompose-fmongodb/compose.yamlup-d`, а структура коллекции и индексов задаётся скриптом `mongodb/initdb/001_init.js`.

3.1 Назначение MongoDB-варианта

MongoDB рассматривается как объектно-центричная альтернатива PostgreSQL. Если в реляционном варианте единицей хранения является отдельная операция, то здесь основной единицей становится документ объекта со встроенным массивом операций.

3.2 Выбранная документная модель

Для текущего этапа принята модель “один объект — один документ со встроенной историей операций”. В текущем варианте используется одна коллекция:

- `objects`.

Документ верхнего уровня содержит:

- `objectId`;
- массив `operations`.

Каждый элемент массива `operations` соответствует библиотечному типу `Operation` и содержит те же поля, что и в реляционном варианте. Таким образом, меняется не состав данных, а единица агрегирования на уровне хранилища.

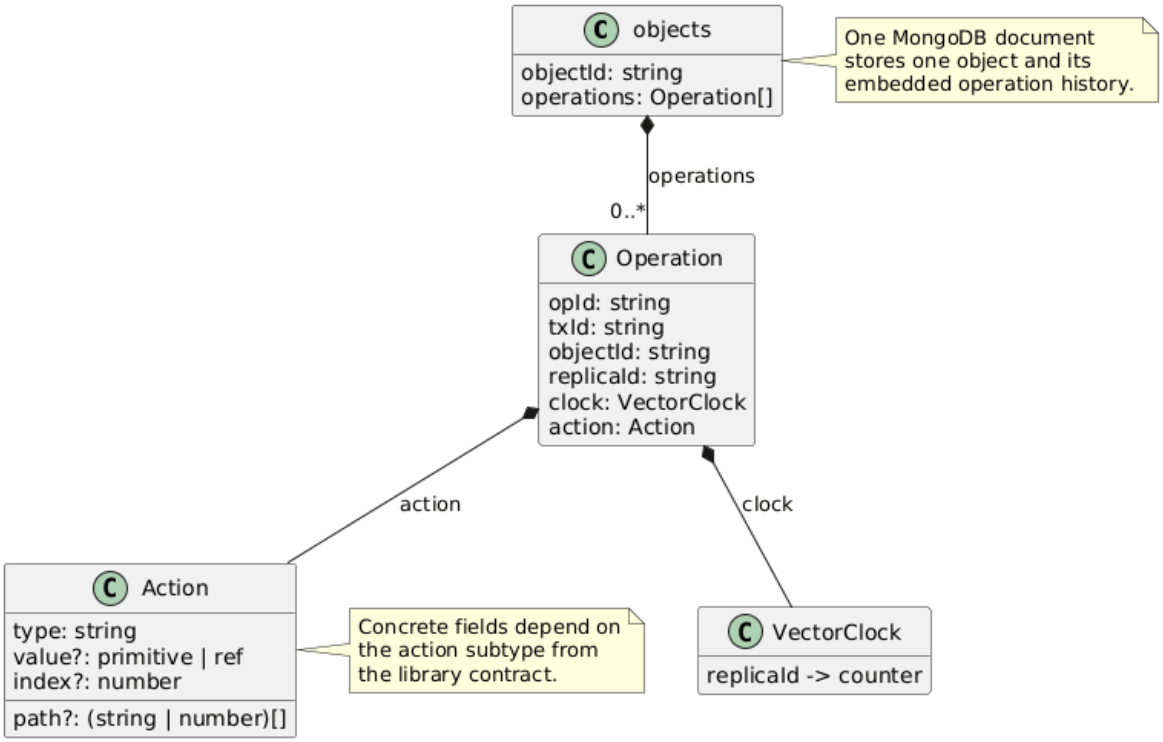

Рисунок 3 – Структура документа объекта в MongoDB

3.3 Индексация и сценарии чтения

В `mongodb/README.md` зафиксированы следующие направления индексации:

- уникальный индекс по `objectId`;
- индекс по `operations.opId`;
- индекс по `operations.replicaId`;
- индекс по `operations.action.type`.

Это хорошо отражает специфику документной модели: индексируются вложенные поля массива операций, а не top-level коллекция операций. Основные сценарии запросов при такой организации следующие:

- поиск документа объекта по `objectId`;
- чтение полной встроенной истории конкретного объекта;

- поиск объекта, содержащего операцию с заданным `opId`;
- поиск объектов, в истории которых есть операции конкретной реплики;
- поиск по значениям `operations.action.type`.

3.4 Преимущества и ограничения

Сильные стороны MongoDB-варианта заключаются в естественном хранении полуструктурированных полей `clock` и `action`, а также в том, что полная история конкретного объекта может быть прочитана из одного документа без отдельного журнала top-level операций.

Ограничения модели также существенны. Размер документа MongoDB ограничен `16 MB`, очень длинная история объекта ухудшает устойчивость embedded-модели, а глобальные выборки по операциям через весь набор документов менее естественны, чем в PostgreSQL. В `README` также отмечено, что при дальнейшем росте истории может потребоваться сегментация документа.

3.5 Фрагмент инициализации MongoDB

Ниже приведён фактический фрагмент скрипта, создающего коллекцию `objects` и основные индексы документного варианта:

```
db = db.getSiblingDB("crdt_lab");

db.createCollection("objects", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["objectId", "operations"],
      properties: {
        objectId: { bsonType: "string" },
        operations: {
          bsonType: "array",
          items: {
```

```
13            bsonType: "object",
14            required: [
15              "opId",
16              "txId",
17              "objectId",
18              "replicaId",
19              "clock",
20              "action"
21            ],
22            properties: {
23              opId: { bsonType: "string" },
24              txId: { bsonType: "string" },
25              objectId: { bsonType: "string" },
26              replicaId: { bsonType: "string" },
27              clock: { bsonType: "object" },
28              action: {
29                bsonType: "object",
30                required: ["type"],
31                properties: {
32                  type: { bsonType: "string" }
33                }
34              }
35            }
36          }
37        }
38      }
39    }
40  }
41 });
42
43 db.objects.createIndex({ objectId: 1 }, { unique: true, name: "uq_object_id"
    });
44 db.objects.createIndex({ "operations.opId": 1 }, { name: "idx_operations_op_id
    " });
45 db.objects.createIndex({ "operations.replicaId": 1 }, { name: "
    idx_operations_replica_id" });
46 db.objects.createIndex({ "operations.action.type": 1 }, { name: "
    idx_operations_action_type" });
```

Листинг 2 – Скрипт инициализации MongoDB

3.6 Наблюдения по экспериментальным данным

Для MongoDB уже зафиксированы характеристики тестового набора данных: `10000` документов коллекции `objects`, от `12` до `1000` операций в одном документе и среднее значение `507.7832` операций на объект. Это показывает, что документная модель исследуется не только концептуально, но и на заметном объёме данных.

ГЛАВА 4. ВАРИАНТ НА YTSAURUS

4.1 Общая характеристика платформы

YTsaurus представляет собой отечественную open-source платформу для распределённого хранения и обработки больших объёмов данных. В отличие от классических СУБД, ориентированных преимущественно либо на транзакционные, либо на аналитические сценарии, YTsaurus изначально проектируется как многофункциональная инфраструктурная платформа, объединяющая распределённое хранилище данных, средства пакетной обработки, механизмы выполнения SQL-подобных запросов и поддержку динамических таблиц.

Архитектурно YTsaurus сочетает несколько уровней. Базовый уровень образует распределённое хранилище и пространство метаданных `Cypress`, в котором данные организуются в виде древовидного пространства имён. Поверх него реализуются механизмы хранения статических и динамических таблиц, а также вычислительная подсистема, поддерживающая `MapReduce`-обработку и более высокоуровневые инструменты, такие как `YQL`, `CHYT` и `SPYT`. Таким образом, YTsaurus следует рассматривать не как отдельную узкоспециализированную базу данных, а как целостную экосистему для хранения, трансформации и анализа данных в распределённой среде.

Практическое назначение YTsaurus связано с задачами, в которых требуется одновременно обеспечить горизонтальную масштабируемость, отказоустойчивость, изолированное сосуществование нескольких нагрузок и поддержку различных способов доступа к данным. Платформа ориентирована на сценарии обработки больших массивов событий, логов, транзакционных записей и иных наборов данных, для которых важны не только объём хранения, но и возможность последующего параллельного анализа.

С точки зрения настоящей работы интерес к YTsaurus обусловлен

тем, что данная платформа предлагает ещё один принципиально отличный контекст хранения данных по сравнению с PostgreSQL и MongoDB. Если PostgreSQL в данном исследовании рассматривается как реляционное хранилище глобального журнала операций, а MongoDB — как документное объектно-центричное хранилище встроенной истории, то YTsaurus представляет собой распределённую платформу, в рамках которой необходимо отдельно определить, какая модель представления operation-based CRDT будет наиболее естественной: журнал операций, табличное представление, ключ-значение или их комбинация.

4.2 Экспериментальное окружение

Практическая часть для YTsaurus выполнялась не в локальном контейнерном окружении, а на реальных кластерах VK, используемых в инфраструктуре рекомендательной платформы.

В отличие от PostgreSQL и MongoDB, где в работе можно было опираться на `EXPLAIN ANALYZE` и `explain("executionStats")`, в рассматриваемом окружении YTsaurus не предоставлял API для аналогичного плана выполнения запросов. Поэтому для YTsaurus удалось замерить только внешнее время исполнения запросов и размер результирующих выборок, без детального разбора внутреннего плана, выбора стратегий чтения и стоимости отдельных стадий выполнения.

4.3 Реализованная модель хранения

В каталоге `YTsaurus/java_files` размещены Java-классы, использовавшиеся для генерации данных и выполнения простых сценариев чтения. Центральным классом доступа к данным является `OperationDao`, работающий с одной dynamic table по пути `//home/dev/n.d.ivanov/operations`. Каждая строка таблицы соответствует одной неизменяемой

операции op-based CRDT.

Содержательно эта модель близка к PostgreSQL-варианту: единицей хранения остается отдельная операция, а не агрегированный документ объекта. В записи сохраняются:

- составной ключ objectId, replicaId, txId, opId;
- поле clock как снимок векторных часов;
- поле action как сериализованное CRDT-действие.

Порядок полей в составном ключе выбран не случайно. Он оптимизирован под типовой доступ по objectId, затем по replicaId и txId. Тем самым YTsaurus-вариант в данной реализации также моделирует глобальный журнал операций, но уже поверх распределённой dynamic table.

4.4 Генератор данных и сценарии запросов

Класс OperationGenerator формирует синтетический набор операций в домене календарных событий. Генерация выполняется для нескольких объектов и реплик, при этом для каждой операции создаётся корректный снимок векторных часов. В тестовой конфигурации использовались:

- 10 реплик;
- 10 объектов;
- 100000 операций на объект.

Итоговый объём составил около 1 000 000 операций, а по данным таблицы фактически было сохранено 1 ̃190 000 строк. Это означает, что в экспериментальном контуре таблица уже содержала объём порядка одного миллиона записей, достаточный для первичных сравнительных замеров.

В классе MainYt реализованы два основных режима работы:

- генерация и пакетная запись операций;
- запуск набора простых benchmark-сценариев чтения.

Проверялись следующие запросы:

- поиск всех операций объекта по `objectId`;
- поиск операций объекта от конкретной реплики;
- поиск операций объекта, реплики и транзакции;
- поиск операций объекта по префиксу пути `action.path` для поля `title`;
- аналогичный поиск по пути `participants`.

Важно, что первые три сценария естественно согласованы с порядком ключа и поэтому соответствуют модели хранения. Напротив, фильтрация по `action.path` в текущей реализации не переносится в хранилище: сначала считываются все операции объекта, а затем выполняется фильтрация в памяти Java-приложения. Следовательно, эти сценарии показывают не столько возможности индексного доступа YTsaurus, сколько стоимость полного чтения истории объекта с последующей прикладной обработкой.

ГЛАВА 5. ИЗМЕРЕНИЯ И РЕЗУЛЬТАТЫ

5.1 Набор команд для анализа

Для обеих СУБД в репозитории сохранены отдельные файлы с фактически использованными командами анализа:

- `postgresql/resources/explain_analyze_commands.md`;
- `mongodb/resources/explain_execution_stats_commands.md`.

Эти файлы фиксируют не только формулировки запросов, но и конкретные параметры, использованные при локальных замерах.

5.2 Результаты PostgreSQL

Для PostgreSQL использовался датасет следующего объёма:

- `operations`: `1,000,000`;
- `objects`: `8`;
- `replicas`: `3`.

Дополнительно была снята статистика физического размера таблиц. Основная нагрузка практически полностью сосредоточена в таблице `operations`: её полный размер составил `393 MB`, из которых `221 MB` приходится на данные таблицы и `171 MB` на индексы. Таблицы `objects` и `replicas` при таком датасете занимают пренебрежимо малый объём — по `32 kB` каждая. Это хорошо согласуется с самой моделью хранения: почти вся стоимость хранения в реляционном варианте связана именно с большим журналом операций и поддерживающими его индексами.

Для измерений были выбраны следующие репрезентативные значения:

- `object_id = 'event-7'`: `126,445` операций;
- `replica_id = 'R2'`: `334,323` операций;
- `tx_id = 'R3:tx:238257'`;
- `op_id = 'R3:238257'`.

Сводные результаты измерений для PostgreSQL вынесены в **Приложение А**.

Показательный пример команды для чтения полной истории объекта:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    op_id,
    tx_id,
    object_id,
    replica_id,
    clock,
    action
FROM operations
WHERE object_id = 'event-7'
ORDER BY op_id;
```

Листинг 3 – Пример EXPLAIN ANALYZE для истории объекта в PostgreSQL

Результаты измерений показывают, что PostgreSQL особенно эффективен в точечных и умеренно селективных сценариях. При этом широкие выборки и сортировка больших подмножеств журнала остаются основным источником стоимости.

5.3 Результаты MongoDB

Для MongoDB использовался набор данных со следующими характеристиками:

- количество документов в `objects`: `10000`;
- минимальное число операций в документе: `12`;

- максимальное число операций в документе: `1000`;
- среднее число операций в документе: `507.7832`;
- оценка суммарного числа встроенных операций: примерно `5,077,832`.

По статистике `collStats` логический размер коллекции `objects` составил `1165.44 MiB`, тогда как физический объём хранения (`storageSize`) оказался равен `223.69 MiB`. Суммарный размер индексов составил `57.85 MiB`, а общий объём коллекции вместе с индексами — `281.54 MiB`. Средний размер одного документа оказался равен примерно `119.34 KiB`. Эти значения показывают, что встроенное хранение длинных историй в документах приводит к значительному логическому объёму данных, однако физически коллекция остаётся достаточно компактной за счёт внутреннего сжатия движка хранения.

Для измерений использовались значения:

- `objectId = "event-3299"`;
- `opId = "R1:365809"`;
- `replicaId = "R5"`;
- `action.type = "field.set"`.

Сводные результаты измерений для MongoDB вынесены в **Приложение Б**.

Показательный пример команды для поиска объекта по вложенному `opId`:

```
db.objects.find(
  { "operations.opId": "R1:365809" },
  { _id: 0, objectId: 1 }
).explain("executionStats")
```

Листинг 4 – Пример explain(”executionStats”) для MongoDB

Полученные результаты подтверждают, что MongoDB лучше всего проявляет себя в объектно-центричных сценариях: поиск объекта, чтение полной истории и доступ к последним операциям выполняются естественно и быстро, тогда как глобальные выборки по операциям остаются менее характерными для этой модели.

5.4 Результаты YTsaurus

Согласно сохранённому логу выполнения, были получены следующие времена:

- `findByObjectId`: `5357 ms`, `100000` строк;
- `findByObjectAndReplica`: `714 ms`, `9957` строк;
- `findByObjectReplicaAndTx`: `136 ms`, `1` строка;
- `findByObjectIdAndPath(title)`: `18114 ms`, `8524` строки;
- `findByObjectIdAndPath(participants)`: `19152 ms`, `30237` строк.

Наблюдаемая картина логична. Чем точнее запрос совпадает с префиксом составного ключа, тем быстрее он выполняется. Самым дешёвым оказывается сценарий чтения одной операции по сочетанию `objectId`, `replicaId` и `txId`. Напротив, сценарии по `path` получаются существенно дороже, поскольку требуют сначала извлечь всю историю объекта, а затем отфильтровать её на прикладном уровне. По сути, это подтверждает, что без отдельного проектирования ключа или дополнительного представления таблицы запросы по вложенному пути действия для YTsaurus в такой схеме оказываются неестественными.

5.5 Параметры таблицы и объём хранения

Скриншоты интерфейса YTsaurus позволяют зафиксировать важные параметры физического хранения таблицы `//home/dev/n.d.ivanov/operations`.

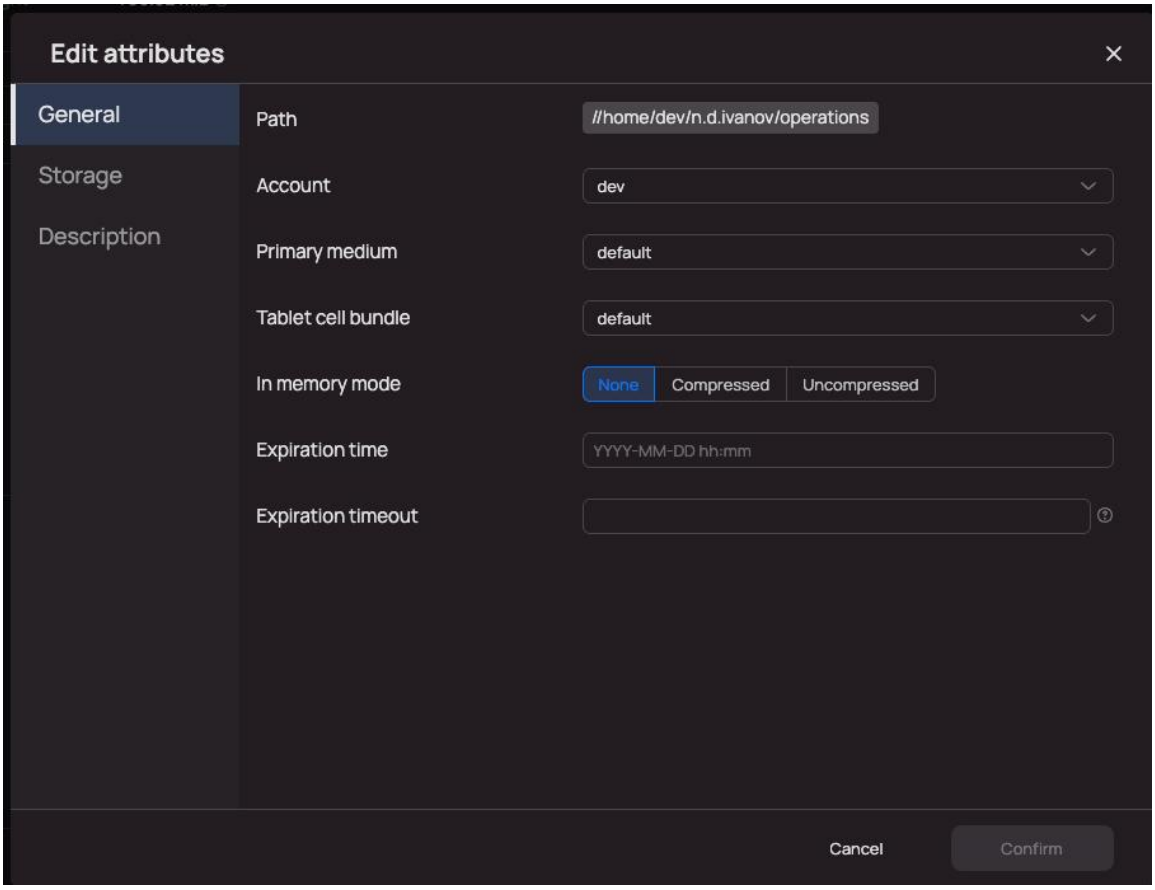

Рисунок 4 – Общие параметры таблицы YTsaurus

Из общих атрибутов видно, что таблица размещена в аккаунте `dev`, использует `primary medium = default`, `tablet cell bundle = default`, а режим `in-memory` отключён. Это означает, что в данном эксперименте таблица не была специально оптимизирована под хранение данных в памяти и оценивалась в более консервативной конфигурации.

На уровне storage-настроек таблица была сконфигурирована как `optimize for lookup`, с кодеком сжатия `lz4` и репликацией `3`. Такое сочетание выглядит оправданным для сценария point lookup и чтения по префиксу ключа: выбран режим, ориентированный именно на запросы по

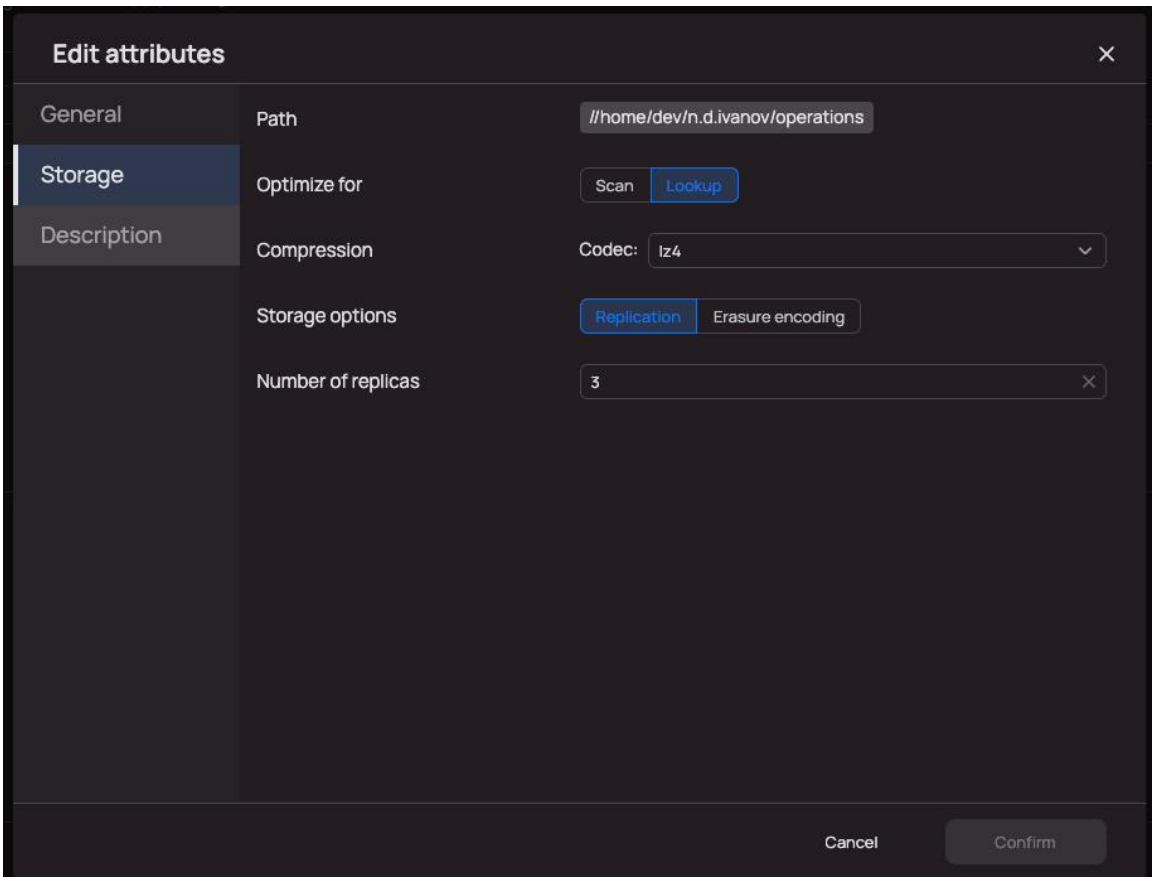

Рисунок 5 – Параметры хранения таблицы YTsaurus

ключу, а не на последовательный scan большого массива данных.

По данным интерфейса таблица содержала примерно `1 190 000` строк, `3` chunk'a, `457.44 MiB` в несжатом виде и `111.68 MiB` в сжатом виде. При трёхкратной репликации суммарное занимаемое дисковое пространство составило `336.20 MiB`, а `data weight` был равен `365.63 MiB`.

Эти значения позволяют сделать несколько выводов. Во-первых, сжатие `lz4` оказалось весьма эффективным: отношение несжатого объёма к сжатому составляет примерно `4.1x`. Во-вторых, итоговый показатель `total disk space` практически совпадает с тройным объёмом сжатых данных, что хорошо согласуется с включённой репликацией `3`. В-третьих, даже при хранении около 1.19 млн операций суммарный физический объём таблицы остаётся умеренным, что делает подобную схему хранения реалистичной для журнала CRDT-операций, по крайней мере на масштабе миллионов записей.

Rows	≈ 1 190 000
Chunks	3
Uncompressed size	457.44 MiB
Compressed size	111.68 MiB
Primary medium	Default
Total disk space	336.20 MiB
Data weight	365.63 MiB

Рисунок 6 – Статистика объёма таблицы YTsaurus

5.6 Промежуточный вывод

Проведённый эксперимент показывает, что YTsaurus может использоваться как распределённое хранилище append-only журнала `op-based CRDT`-операций, если ключ проектируется под наиболее частые сценарии чтения. В такой схеме платформа естественно поддерживает выборки по `objectId`, `replicaId` и `txId`, однако запросы по содержимому вложенного `action.path` без дополнительной перестройки модели оказываются заметно менее эффективными. При этом отсутствие доступа к `EXPLAIN`-подобным механизмам в используемом кластере ограничивает анализ уровнем внешнего времени исполнения и не позволяет детально разобрать внутреннюю стратегию выполнения запросов.

ЗАКЛЮЧЕНИЕ

В работе были рассмотрены три варианта хранения `op-based CRDT`-операций: реляционный на PostgreSQL, документный на MongoDB и распределённый табличный вариант на YTsaurus. Для каждого из подходов была проанализирована единица хранения, структура данных, характер типовых запросов и практические ограничения, возникающие при работе с журналом операций.

PostgreSQL показал себя как наиболее естественный выбор для модели глобального immutable operation log, в которой операция рассматривается как самостоятельная запись, а важную роль играют ссылочная целостность, прозрачность схемы и возможность детального анализа выполнения запросов. MongoDB, напротив, оказался более естественным для объектно-центричного хранения, где история операций агрегируется внутри документа и может быть быстро прочитана целиком для одного объекта. YTsaurus в исследованной постановке продемонстрировал, что также может использоваться для хранения большого append-only журнала операций, если составной ключ проектируется в соответствии с реальными сценариями доступа.

Проведённое сравнение показывает, что выбор хранилища для `op-based CRDT` не сводится только к сравнению производительности отдельных запросов. На практике определяющими оказываются архитектурная модель хранения, характер доступа к истории операций, требования к масштабируемости и доступные средства наблюдаемости. В этом смысле PostgreSQL, MongoDB и YTsaurus представляют три разных подхода к хранению одной и той же прикладной структуры данных, и каждый из них оказывается оправданным в собственном классе сценариев.

Если рассматривать единицу хранения, то PostgreSQL и YTsaurus в данной работе оказались ближе друг к другу, чем к MongoDB. В обоих случаях операция является самостоятельной записью, пригодной для выборок по

идентификаторам объекта, реплики и транзакции. Однако различие между ними принципиально состоит в том, что PostgreSQL даёт развитые механизмы реляционного анализа, прозрачные планы выполнения запросов и привычную схему ограничений целостности, тогда как YTsaurus сильнее ориентирован на распределённое хранение и доступ по ключу в масштабируемой инфраструктурной среде.

MongoDB, напротив, реализует иную модель: операция не существует как отдельный top-level элемент хранения, а включается во встроенный массив внутри документа объекта. Это даёт естественное преимущество в сценариях чтения полной истории одного объекта, но делает менее удобными глобальные выборки по операциям через весь набор данных.

С точки зрения запросов различие также проявилось достаточно явно. PostgreSQL демонстрирует сильные результаты в точечных и умеренно селективных сценариях, а также предоставляет детальную диагностику через `EXPLAIN ANALYZE`. MongoDB хорошо поддерживает доступ к истории конкретного объекта и поиск по вложенным полям операций за счёт multikey-индексов. YTsaurus в исследованной конфигурации показал, что лучше всего подходит для запросов, согласованных с порядком составного ключа, то есть прежде всего по `objectId`, `replicaId` и `txId`, тогда как фильтрация по содержимому `action.path` без специального проектирования модели оказывается существенно менее эффективной.

Отдельно следует отметить различие в глубине экспериментального анализа. Для PostgreSQL и MongoDB удалось исследовать не только время, но и сами планы выполнения запросов. Для YTsaurus на доступном кластере VK такой возможности не было: измерялись только внешние времена исполнения и размеры результатов. Поэтому выводы по YTsaurus неизбежно имеют более прикладной и менее плано-ориентированный характер.

При этом с точки зрения самой прикладной логики работы с композитными `CRDT`-объектами наиболее естественным в рамках данного исследова-

ния оказался именно MongoDB-подход. Причина состоит в том, что в документной модели исследователь и разработчик оперируют прежде всего объектом как целостной сущностью, а не множеством отдельных строк журнала. История изменений, относящаяся к одному `objectId`, оказывается собрана в пределах одного документа и потому воспринимается как единый агрегат, близкий к исходному прикладному представлению календарного события.

Такой способ хранения лучше соответствует интуитивной модели предметной области. При работе с operation-based `CRDT` на прикладном уровне обычно важно мыслить не отдельными операциями как таковыми, а состоянием и эволюцией конкретного объекта, над которым эти операции выполняются. В MongoDB эта перспектива поддерживается самой структурой хранилища: объект и его история оказываются связаны непосредственно, без необходимости каждый раз восстанавливать эту связь через внешний журнал операций. В результате модель оказывается проще для концептуального восприятия, ближе к объектному устройству приложения и удобнее для сценариев, где основной единицей анализа является именно отдельный CRDT-объект.

Именно поэтому, хотя PostgreSQL предоставляет более сильные средства анализа, строгую схему и удобную работу с глобальным operation log, а YTsaurus интересен как распределённая масштабируемая платформа, MongoDB в данной постановке можно считать наиболее естественным вариантом с точки зрения представления самой предметной сущности. Он лучше отражает объектную природу исследуемых данных и позволяет рассматривать историю операций не как внешнее множество событий, а как внутреннюю характеристику конкретного объекта.

ПРИЛОЖЕНИЕ А. СВОДНЫЕ РЕЗУЛЬТАТЫ POSTGRESQL

Таблица 1 – Ключевые результаты EXPLAIN ANALYZE для PostgreSQL

Сценарий	**План**	**Время**	**Наблюдение**
Размер таблицы `operations`	Таблица `operations` + индексы	`393 MB`	`221 MB` данных и `171 MB` индексов; основной объём хранения сосредоточен в журнале операций
История объекта по `object_id`	`Bitmap Index Scan` + `Parallel Bitmap Heap Scan` + `Gather Merge`	`573.7 ms`	Индекс по `object_id` используется, но значимая часть стоимости уходит в чтение heap pages и сортировку
Операции транзакции по `tx_id`	`Index Scan` по `idx_operations_tx_id`	`0.245 ms`	Быстрый точечный сценарий с эффективным использованием B-tree индекса
Операции реплики по `replica_id`	`Parallel Seq Scan` + сортировка	`210.0 ms`	Индекс не выбран из-за низкой селективности условия
Поиск по `op_id`	`Index Scan` по `operations_pkey`	`0.213 ms`	Первичный ключ полностью закрывает данный сценарий
Фильтрация по частому `action.type = field.set`	`Parallel Seq Scan`	`1004.4 ms`	Индекс невыгоден для очень частого значения
Фильтрация по селективному `action.type = set.remove`	`Bitmap Index Scan` + `Parallel Bitmap Heap Scan`	`275.4 ms`	Индекс по типу действия работает при селективном значении
Поиск по фрагменту `action`	`Bitmap Index Scan` по GIN	`485.2 ms`	GIN-индекс полезен для containment-запросов по `JSONB`
Поиск по фрагменту `clock`	`Bitmap Index Scan` по GIN	`0.534 ms`	Индекс по `clock` реально используется и эффективен

ПРИЛОЖЕНИЕ Б. СВОДНЫЕ РЕЗУЛЬТАТЫ MONGODB

Таблица 2 – Ключевые результаты executionStats для MongoDB

Сценарий	**План**	**Показатели**	**Наблюдение**
Размер коллекции `objects`	`collStats`	`1165.44 MiB` логически, `223.69 MiB` storage, `57.85 MiB` индексы	При `10000` документах и среднем размере документа `119.34 KiB` физический объём остаётся умеренным за счёт сжатия
Поиск объекта по `objectId`	`IXSCAN -> FETCH`	`1 key / 1 doc`	Базовый сценарий полностью поддержан уникальным индексом
Чтение полной истории объекта	`IXSCAN -> FETCH`	`1 key / 1 doc`, `2 ms`	История объекта читается как один документ
Поиск по `operations.opId`	`IXSCAN -> FETCH -> PROJECTION`	`1 key / 1 doc`	Multikey индекс работает эффективно
Поиск по `operations.replicaId`	`IXSCAN -> FETCH -> LIMIT`	`100 keys / 100 docs`	Индекс используется корректно, но сценарий вторичен для embedded-модели
Поиск по `operations.action.type`	`IXSCAN -> FETCH -> LIMIT`	`100 keys / 100 docs`	Индекс по типу действия работает корректно
Чтение последних операций через `$slice`	`IXSCAN -> FETCH -> PROJECTION`	`1 key / 1 doc`, `4 ms`	Сценарий хорошо соответствует объектной модели

ПРИЛОЖЕНИЕ В. СВОДНЫЕ РЕЗУЛЬТАТЫ YTSAURUS

Таблица 3 – Ключевые результаты измерений для YTsaurus

Сценарий	**Условие**	**Время**	**Результат**	**Наблюдение**
Все операции объекта	`objectId = event-0`	`5357 ms`	`100000` строк	Чтение большой истории одного объекта по первому столбцу составного ключа; сценарий естественен для выбранной схемы, но остаётся дорогим из-за объёма результата
Операции объекта конкретной реплики	`objectId = event-0`, `replicaId = replica-1`	`714 ms`	`9957` строк	Существенно быстрее за счёт более точного совпадения с префиксом ключа
Операция по объекту, реплике и транзакции	`objectId = event-0`, `replicaId = replica-1`, `txId = replica-1:tx:1000`	`136 ms`	`1` строка	Наиболее селективный сценарий; лучше всего соответствует key-lookup модели таблицы
Операции объекта по пути `title`	`objectId = event-0`, фильтрация по `action.path`	`18114 ms`	`8524` строки	Поиск по пути не поддерживается напрямую ключом: сначала считывается вся история объекта, затем выполняется фильтрация в приложении
Операции объекта по пути `participants`	`objectId = event-0`, фильтрация по `action.path`	`19152 ms`	`30237` строк	Самый дорогой сценарий из-за полного чтения истории и последующей прикладной фильтрации по пути

ПРИЛОЖЕНИЕ Г. СРАВНЕНИЕ ПОДХОДОВ ХРАНЕНИЯ

Таблица 4 – Сравнение вариантов хранения

Критерий	**PostgreSQL**	**MongoDB**	**YTsaurus**
Единица хранения	Отдельная операция	Документ объекта со встроенной историей	Отдельная операция в distributed dynamic table
Объём хранения	`operations`: `393 MB` total, из них `221 MB` данные и `171 MB` индексы	`objects`: `1165.44 MiB` logical size, `223.69 MiB` storage, `57.85 MiB` индексы	`1 190 000` строк, `457.44 MiB` uncompressed, `111.68 MiB` compressed, `336.20 MiB` total disk space
Физическая структура	Таблицы `replicas`, `objects`, `operations`	Коллекция `objects` и массив `operations`	Dynamic table `//home/dev/n.d.ivanov/operations` с составным ключом `objectId, replicaId, txId, opId`
Сильная сторона	Глобальный журнал операций, аналитика, ссылочная целостность	Быстрое чтение полной истории объекта	Масштабируемое распределённое хранение и эффективный lookup по префиксу ключа
Слабая сторона	Дорогая сортировка больших выборок, часть структуры в `JSONB`	Ограничение размера документа и менее естественные глобальные выборки	Ограниченная наблюдаемость плана выполнения и слабая естественность запросов по `action.path` без отдельного проектирования модели
Инструмент анализа	`EXPLAIN ANALYZE`	`explain("executionStats")`	Замер внешнего времени выполнения без `EXPLAIN`-API в доступном контуре